

파이썬 최고의 라이브러리 : 넘파이 KarL



NumPy

파이썬 리스트와 넘파이

- 데이터 과학을 위한 많은 라이브러리 중에서 가장 기본이 되는 라이브러리가 바로 **넘파이**Numpy이다.
- 2020년 9월 저명한 과학 저널 **네이처**Nature에 넘파이에 대한 리뷰 논문이 게재 되었다.
- 논문은 **텐서**tensor라 불리는 다차원 배열을 효율적으로 다루는 넘파이가 과학 분야에 파이썬 생태계를 제공했고, 여러 과학 분야의 수치 데이터 처리 기술들이 상호운용성을 가질 수 있도록 하는 역할을 했다고 밝히고 있다.
- 넘파이는 데이터를 다차원 배열에 저장할 때 같은 자료형으로만 저장할 수 있다. 이렇게 같은 자료형으로만 데이터를 저장하면 각 데이터 항목에 필요한 저장공간이 일정하다. 따라서 몇 번째 위치에 있는 항목이든 그 순서만 안다면 간단한 계산으로 바로 접근할 수 있는 것이다.



파이썬 리스트와 넘파이

- 파이썬의 리스트는 여러 자료형의 값들을 하나의 변수에 저장할 수 있는 자료구조로서 강력하고 활용도가 높다.
- 하지만 데이터를 처리할 때는 리스트와 리스트 간의 다양한 연산이 필요한데, 이러한 점에서 기능이 다소 부족하고, 연산 속도도 빠르지 않다. 따라서 데이터 과학자들은 넘파이 배열을 선호한다.



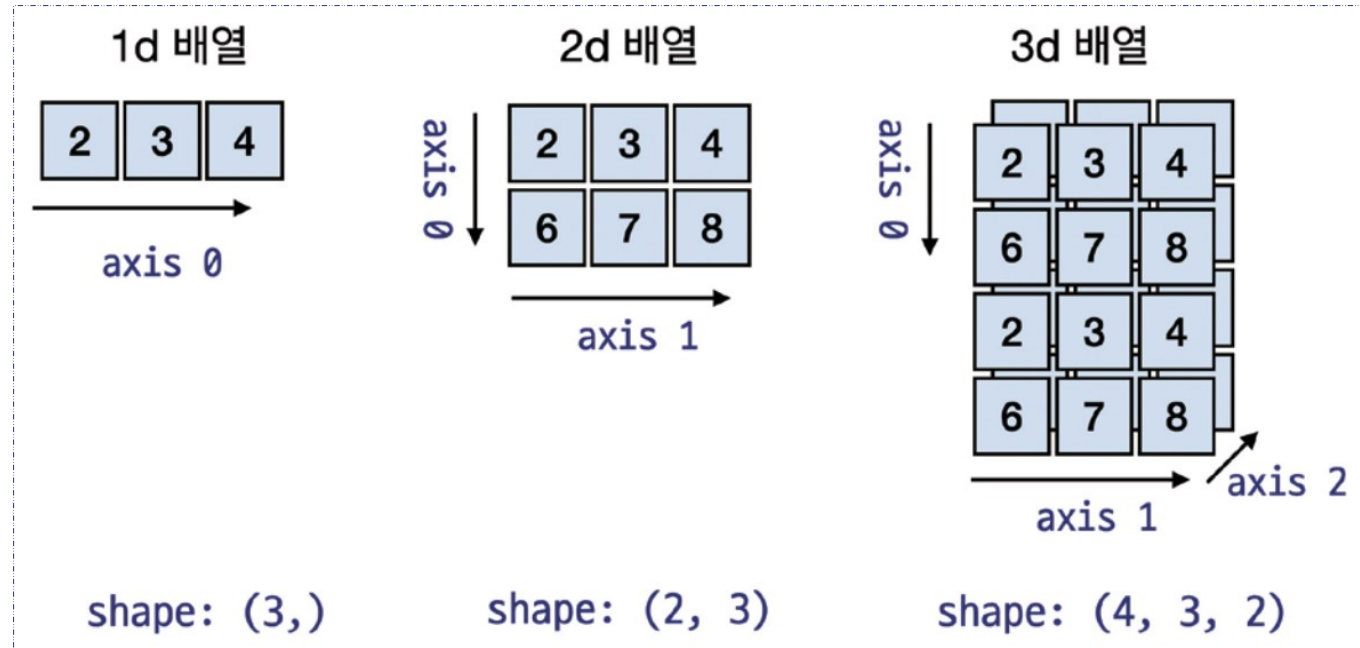
```
import numpy as np      # numpy의 별칭으로 np를 지정함
```

```
a = np.array([2, 3, 4]) # 다차원 배열 객체
```

- 넘파이를 사용할 때는 반드시 numpy 모듈을 import 해야만 한다. 일반적으로 넘파이 모듈에 대해서는 다음과 같이 as 키워드 다음에 나오는 np라는 별칭을 지정하여 사용한다.
- 넘파이의 가장 기본이 되는 다차원 배열을 array() 함수와 리스트를 이용해서 위와 같이 생성해 보도록 하자.

파이썬 리스트와 넘파이

- 이 배열은 2, 3, 4의 정수 값으로 구성되어 있는데 이 개별적인 값을 **항목**item 또는 **원소**element라고 한다.
- 이 배열 내부의 각 요소는 **인덱스**index라고 불리는 정수들로 참조된다.
- 또한 넘파이에서 차원은 **축**axis이라고도 한다.
- 위의 그림은 넘파이의 1차원, 2차원, 3차원 배열의 축과 그 **형상**shape을 나타내고 있다.
- 그림의 가장 오른쪽에 있는 배열은 형상이 (4, 3, 2)와 같은 형태로 표시되는데 이 형상의 의미는 3개의 튜플 형식으로 표현되는 **3차원 배열 내부의 각 축이 가지는 최대 원소의 수**를 나타낸다.



파이썬 리스트 vs 넘파이 다차원 배열

- 리스트와 넘파이의 가장 큰 차이는 계산 성능이다.
- 넘파이는 대용량의 배열과 행렬 연산을 수행하는 고차원적인 수학 연산자와 함수를 포함하고 있으며, 성능이 우수한 ndarray 객체를 제공한다.
- **배열array**은 동일한 자료형을 가진 데이터를 연속으로 저장한다.
- **파이썬의 리스트는 동일하지 않은 자료형을 가진 항목들을 담을 수 있다.** 이 때문에 파이썬의 리스트를 사용하면 원소에 접근하는 속도가 매우 느리다.
- 넘파이의 **ndarray 객체는 동일한 자료형의 항목들만 저장**한다. 이러한 차이로 넘파이가 원소에 접근하는 속도가 더 빠르다.

파이썬 리스트 vs 넘파이 다차원 배열

- 리스트는 데이터를 벡터의 개념으로 해석하지 않고 단순히 여러 데이터가 나열된 것으로만 생각한다.
- 넘파이는 배열을 선형대수의 벡터 개념으로 다룬다.
- 이 차이는 리스트와 넘파이 배열에 대해 덧셈 연산을 적용해서 확인해 보자. 동윤이는 두 군데 매장 A와 B를 소유하고 있다고 하고, 각 매장의 지난 3개월 매출이 다음과 같이 store_a와 store_b라는 리스트에 저장되었다고 하자.



```
store_a = [20, 10, 30] # 매장 A의 매출 : 파이썬 리스트로 표현  
store_b = [70, 90, 70] # 매장 B의 매출 : 파이썬 리스트로 표현
```

파이썬 리스트 vs 넘파이 다차원 배열

- 두 리스트를 합하면 어떻게 될까? 아래와 같이 두 리스트는 월별로 합산되지 않고, 연결되어 버린다.



```
list_sum = store_a + store_b    # 파이썬 리스트의 더하기 연산  
list_sum
```

```
[20, 10, 30, 70, 90, 70]
```

- 리스트를 이루는 원소들의 인덱스가 특별한 의미를 갖지 않고 단순히 하나의 리스트 내에서의 순서만을 나타내는 것으로 간주하기 때문이다.
- 이와는 반대로, 넘파이는 대응되는 인덱스들이 더해지고 뺄 수 있는 연관된 데이터라고 가정한다. 데이터를 **벡터**vector로 간주하는 것이다.

파이썬 리스트 vs 넘파이 다차원 배열

- 실제로 넘파이의 다차원 배열 덧셈을 수행해 보자. 넘파이의 ndarray는 파이썬의 리스트를 이용하여 쉽게 만들 수 있다. 넘파이가 가진 `array()` 함수에 파이썬 리스트를 넣으면 넘파이 배열을 돌려준다.



```
np_store_a = np.array(store_a) # store_a 리스트를 넘파이 배열로 변환  
np_store_b = np.array(store_b) # store_b 리스트를 넘파이 배열로 변환
```

- 이 두 배열을 더해서 결과를 확인해 보자. 매달 발생한 두 매장의 매출을 월별로 더한 의미 있는 결과를 얻을 수 있을 것이다.



```
array_sum = np_store_a + np_store_b # 넘파이 배열의 덧셈 결과를 확인하자  
array_sum
```

```
array([ 90, 100, 100])
```

다차원 배열의 속성들

- 넘파이의 핵심이 되는 **다차원 배열** `ndarray`은 다음과 같은 속성을 가지고 있다.
- 아래와 같이 간단한 값을 가지는 다차원 배열 객체 `a`를 만들고 이 객체의 속성을 출력해 보자. 우리는 이러한 속성을 이용하여 프로그램의 오류를 찾거나 배열의 상세한 정보를 손쉽게 조회할 수 있다.



```
a = np.array([2, 3, 4])      # 넘파이 ndarray 객체의 생성
```

```
# a 객체의 형상(shape), 차원, 요소의 자료형, 요소의 크기(byte), 요소의 수  
a.shape, a.ndim, a.dtype, a.itemsize, a.size
```

```
((3,), 1, dtype('int64'), 8, 3)
```

다차원 배열의 속성들

- 이 배열 `a`는 다음과 같은 속성을 가지는데, 이 속성의 자세한 설명은 다음과 같다.

속 성	설 명
<code>ndim</code>	배열 축 혹은 차원의 개수를 나타냄.
<code>shape</code>	배열의 형상을 기술하며 (m, n) 형식의 튜플 형으로 나타냄.
<code>size</code>	배열 원소의 개수를 말하며 (m, n) 형상 배열의 size는 $m \cdot n$ 임.
<code>dtype</code>	배열 원소의 자료형을 기술함.
<code>itemsize</code>	배열 원소의 크기를 바이트 단위로 기술함. 예를 들어 <code>int32</code> 자료형의 크기는 $32/8 = 4$ 바이트가 됨.
<code>data</code>	배열의 실제 원소를 포함하고 있는 버퍼임.
<code>stride</code>	배열의 차원별로 다음 요소로 점프하는 데에 필요한 거리를 바이트로 표시한 값.

다차원 배열의 속성들

- 여러 가지 속성 중에서 다차원 배열의 행과 열을 비롯한 차원의 형상을 출력하는 **shape** 속성은 넘파이 이용자들이 가장 자주 사용하는 속성이다.
- a 배열은 (2, 3, 4)의 원소를 가지는 일차원 배열이기 때문에 그 shape 값이 (3,) 과 같이 나타나며, 다음과 같은 2행 3열의 형태를 가진 2차원 배열 b의 속성은 (2, 3)으로 나타난다.



```
b = np.array([[1, 2, 3], [4, 5, 6]]) # 넘파이 ndarray 객체의 b 생성  
b.shape      # 2행 3열의 배열
```

```
(2, 3)
```

- 다차원 배열은 파이썬 리스트와 유사하게 + 나 * 연산자와 같은 연산자를 적용할 수 있다. 하지만 이 연산자들이 하는 일은 리스트와는 다르다.

다차원 배열의 속성들

- 예를 들어서, 다음과 같은 값을 가진 `a`, `b`라는 `ndarray` 객체가 있을 때 이 두 객체에 사칙연산을 적용해 보도록 하자.



```
a = np.array([10, 20, 30])    # 넘파이 ndarray 객체 a 생성  
b = np.array([1, 2, 3])      # 넘파이 ndarray 객체 b 생성  
a + b
```

```
array([11, 22, 33])
```



```
a - b
```

```
array([ 9, 18, 27])
```

다차원 배열의 속성들



`a * b`

```
array([10, 40, 90])
```



`a / b`

```
array([10., 10., 10.])
```

- 이 결과에서 알 수 있는 것은 사칙연산을 수행할 때 개별 원소별로 덧셈, 뺄셈, 곱셈 나눗셈이 이루어진다는 것이다.
- 따라서 a의 10 원소와 b의 1 원소가 사칙연산을 수행하여 다차원 배열의 첫 원소 11(10 + 1의 결과), 9(10 - 1의 결과), 10(10 * 1의 결과), 10.(10 / 1의 결과)이 됨을 알 수 있다.

다차원 배열과 브로드캐스팅

- 다음과 같이 a 배열을 생성하여 스칼라값 10을 곱하면 a의 모든 원소에 대하여 10이 곱해져서 결과가 100, 200, 300이 되는 것을 확인할 수 있다.
- 덧셈에 대해서도 동일하게 원소별 덧셈을 하는 것도 확인할 수 있다.



```
a = np.array([10, 20, 30])  
a * 10
```

```
array([100, 200, 300])
```

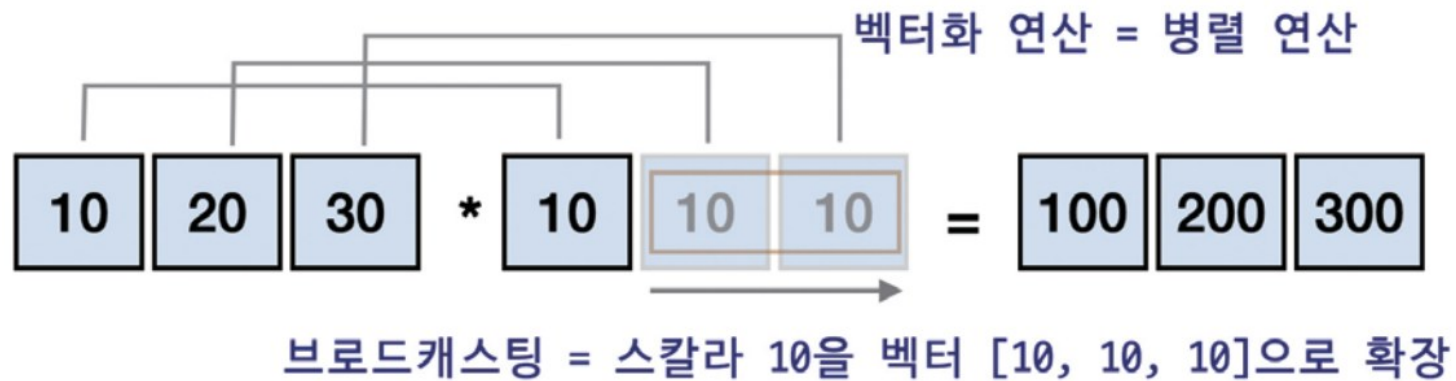


```
a + 10
```

```
array([20, 30, 40])
```

다차원 배열과 브로드캐스팅

- 넘파이는 $a * 10$ 을 수행할 때 `np.array([10, 20, 30]) * np.array([10, 10, 10])`와 같이 a 의 차원에 맞게, 그림과 같이 스칼라 10을 벡터로 확장시켜주는 작업을 하는데 이를 **브로드캐스팅** `broadcasting`이라고 한다.
- 브로드캐스팅과 동시에 파이썬은 하나의 명령을 여러 데이터에 적용하여 병렬적으로 연산하는데 이것을 **벡터화 연산** `vectorized operation`이라고도 한다.



다차원 배열과 브로드캐스팅

- 다음과 같이 2차원 배열 `b`를 만들고 이 배열에 1차원 배열 `c`를 더하고, 곱할 경우 어떤 결과가 나올지 알아보자.



```
b = np.array([[10, 20, 30],  
              [40, 50, 60]])  
c = np.array([2, 3, 4])  
b + c
```

```
array([[12, 23, 34],  
       [42, 53, 64]])
```



```
b * c
```

```
array([[ 20,  60, 120],  
       [ 80, 150, 240]])
```

다차원 배열과 브로드캐스팅

- 이 코드의 수행 결과는 그림과 같이 [2, 3, 4] 라는 값을 가진 배열 c가 같은 값을 가지는 행을 하나 더 생성하여 [[2, 3, 4], [2, 3, 4]] 배열로 변환된 후 [[10, 20, 30], [40, 50, 60]] 과의 덧셈을 병렬적으로 수행한다.
- 두 배열 덧셈의 결과는 [[12, 23, 34], [42, 53, 64]]가 된다.

10	20	30
40	50	60

 +

2	3	4
2	3	4

 =

12	23	34
42	53	64

브로드캐스팅

다차원 배열과 브로드캐스팅

- 넘파이는 다차원 배열을 쉽게 생성하는 함수도 지원하고 있다. `zeros((n, m))` 함수는 $n \times m$ 배열을 생성하는데 그 초기값은 0으로 해준다.
- 만약에 `ones((n, m))` 함수를 호출하면 초기값을 1로 가지는 $n \times m$ 배열을 생성할 수 있다.
- 초기값을 임의의 수 x 로 지정하고 싶을 경우 `full((n, m), x)`이라고 하는 함수를 호출하면 된다.
- 행과 열의 크기가 동일한 정방행렬 중에서 대각선 성분이 1이고 나머지 성분은 0인 행렬을 **단위행렬** `identity matrix`이라고 하는데, `eye(n)` 함수를 이용하면 $n \times n$ 크기의 단위행렬을 생성할 수 있다.

다차원 배열과 브로드캐스팅

 `np.zeros((2, 3))` # 2행 3열의 행렬 생성 시 모든 값을 0으로

```
array([[0., 0., 0.],  
       [0., 0., 0.]])
```

 `np.ones((2, 3))` # 2행 3열의 행렬 생성 시 모든 값을 1로

```
array([[1., 1., 1.],  
       [1., 1., 1.]])
```

 `np.full((2, 3), 100)` # 2행 3열의 행렬 생성 시 모든 값을 100으로

```
array([[100, 100, 100],  
       [100, 100, 100]])
```

 `np.eye(3)` # 3x3 크기의 단위행렬 생성

```
array([[1., 0., 0.],  
       [0., 1., 0.],  
       [0., 0., 1.]])
```

다차원 배열과 브로드캐스팅



다차원 배열 생성하기

난이도 중

대각 성분이 모두 10인 4×4 크기의 다차원 배열을 `eye()` 함수와 브로드캐스팅 기능을 이용하여 생성하여라.

```
array([[10.,  0.,  0.,  0.],
       [ 0., 10.,  0.,  0.],
       [ 0.,  0., 10.,  0.],
       [ 0.,  0.,  0., 10.]])
```

연속적인 값을 가지는 다차원 배열의 생성

- 다음으로는 숫자의 열을 생성하는 함수 `arange(m, n)`에 대해서 알아보자.
- 이 함수는 리스트의 `range()` 함수와 유사하며, `m`부터 `n`미만까지 연속적인 정수나 실수를 만들어준다.
- 이 함수를 호출할 때 `arange(m, n, step)`과 같이 3개의 인자를 넣어주게 되면 `m`부터 `n`까지 `step`만큼 건너뛰며 연속적인 정수나 실수를 생성한다.

연속적인 값을 가지는 다차원 배열의 생성

- 먼저 `arange(0, 10)`을 실행하면 다음과 같은 다차원 배열 객체가 생성된다. 이 객체는 0부터 9까지의 값을 원소로 가진 배열을 생성한다.
- 이제 `arange(0, 10, 2)`과 같이 스텝 값 2를 추가해 주면 0부터 2씩 건너뛰면서 원소를 생성한다.
- 따라서 `arange(0, 10, 2)`는 `[0, 2, 4, 6, 8]`의 값을 가지는 배열을 생성한다. 넘파이의 스텝 값으로는 2뿐만 아니라 다른 값도 넣을 수 있는데 이 값은 정수 자료형이 아니어도 가능하다.

`arange(0, 10)`



`arange(0, 10, 2)`



연속적인 값을 가지는 다차원 배열의 생성

- 그러므로, 아래 마지막 코드와 같이 `arange(0.0, 1.0, 0.2)`와 같은 코드의 실행이 가능한 것이다.

 `np.arange(0, 10)` # 0에서 9까지 연속적인 수열을 생성

`array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])`

 `np.arange(0, 10, 2)` # 2씩 증가하는 수열을 생성

`array([0, 2, 4, 6, 8])`

 `np.arange(0, 10, 3)` # 3씩 증가하는 수열을 생성

`array([0, 3, 6, 9])`

 `np.arange(0.0, 1.0, 0.2)` # 0.2씩 증가하는 수열을 생성

`array([0. , 0.2, 0.4, 0.6, 0.8])`

연속적인 값을 가지는 다차원 배열의 생성

- 넘파이의 `linspace()` 함수는 지정된 구간에서 동일한 간격의 값을 지정된 횟수만큼 생성하는 기능이 있다.



연속적인 값을 가지는 다차원 배열의 생성

- 예를 들어 `np.linspace(0, 10, 5)` 함수는 0에서 10 사이의 구간에 대해(시작 값 0, 마지막 값 10을 포함) 5개의 값을 같은 간격으로 생성하므로 0.0, 2.5, 5.0, 7.5, 10.0의 값 5개를 생성한다.
- 이때 구간의 간격은 동일하며 지정된 개수로 자동으로 계산된다. 만일 값의 개수를 4개로 줄일 경우 0.0, 3.33, 6.66, 10.0의 값 4개를 생성한다.



```
np.linspace(0, 10, 5)
```

```
array([ 0. , 2.5, 5. , 7.5, 10. ])
```

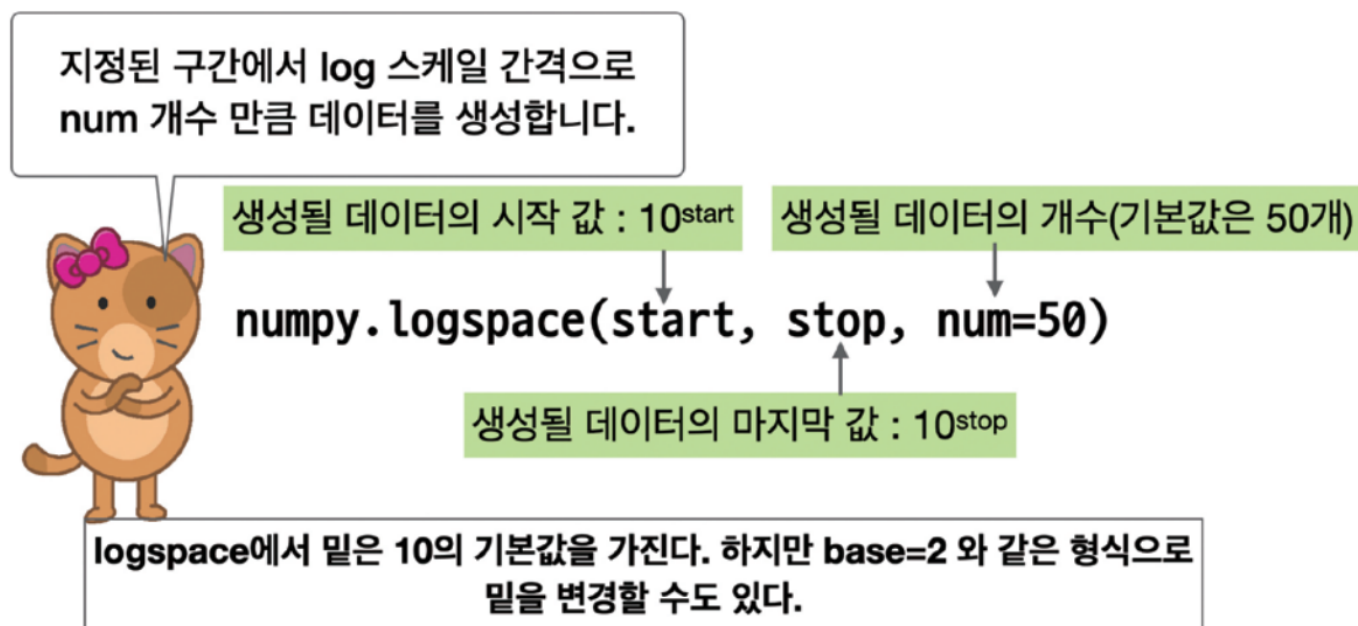


```
np.linspace(0, 10, 4)
```

```
array([ 0. , 3.33333333, 6.66666667, 10. ])
```

연속적인 값을 가지는 다차원 배열의 생성

- 또 다른 함수로 `logspace()` 가 있다. 이것은 로그 스케일로 수들을 생성한다.
- 형식은 `logspace(x, y, n)`이며, 생성되는 수의 시작은 10^x 부터 10^y 까지가 되며, n 개의 수가 생성된다. 수들 사이의 간격은 로그 스케일로 균등하게 결정된다.



연속적인 값을 가지는 다차원 배열의 생성

- 로그 스케일로 균등한 간격의 의미는 아래 코드와 같이 1, 10, 100, 1000, ...의 값에 대한 로그값이 0, 1, 2, 3, ...으로 균등하다는 의미이다.



```
a = np.logspace(0, 5, 6).round(10) # 10^0에서 10^5까지 6개의 수를  
a                                # 로그 스케일 균일한 간격으로 생성한다.
```

```
array([1.e+00, 1.e+01, 1.e+02, 1.e+03, 1.e+04, 1.e+05])
```



```
np.log10(a)    # a 배열에 로그를 취하면 0, 1, 2, 3, 4, 5가 생성됨
```

```
array([0., 1., 2., 3., 4., 5.])
```

연속적인 값을 가지는 다차원 배열의 생성



사인값 출력하기

난이도 중

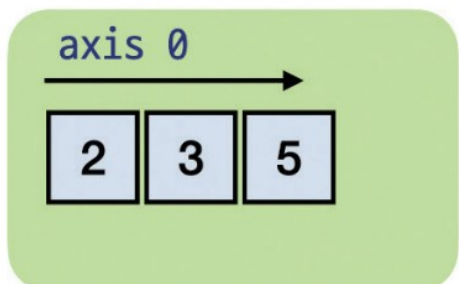
sin0도에서 sin180도 사이의 값 11개를 다음과 같이 출력하여라. 이때 `sin()` 함수의 입력을 라디안을 사용하여라(힌트: 넘파이의 `linspace()` 함수를 사용해보자, 라디안으로 180도는 `np.pi`를 사용하면 된다).

```
[0.00000000e+00  3.09016994e-01  5.87785252e-01  8.09016994e-01
 9.51056516e-01  1.00000000e+00  9.51056516e-01  8.09016994e-01
 5.87785252e-01  3.09016994e-01  1.22464680e-16]
```

다차원 배열의 축과 삽입

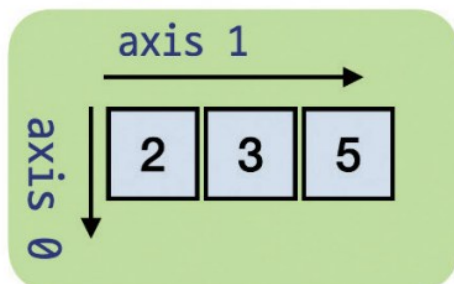
- 다차원 배열의 형상 값에서 (3,)와 (1, 3), 그리고 (3, 1)의 차이를 제대로 이해하는 것은 중요하다.
- 여기서 (3,)은 축이 하나인 1차원 배열이고, (1, 3), (3, 1)은 모두 2차원 배열이다.
- 2차원 배열은 [[로 시작하여]]로 표시된다는 점에 유의하자.

1d 배열 : shape = (3,)



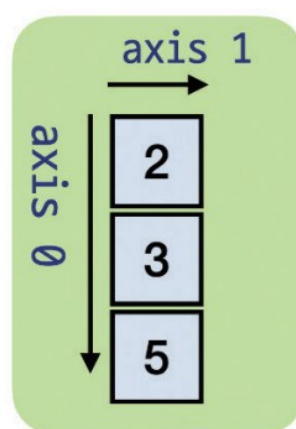
`np.array([2, 3, 5])`

2d 배열 : shape = (1, 3)



`np.array([[2, 3, 5]])`

2d 배열 : shape = (3, 1)



`np.array([[2], [3], [5]])`



형상과 축은 넘파이에서 아주 중요하답니다.

다차원 배열의 축과 삽입

- 다차원 배열에서 축을 이용하여 원소를 삽입하는 방법에 대하여 알아보자.
- 다음과 같은 다차원 배열 `a`가 있을 때, 이 배열의 두 번째 원소로 2를 삽입하고자 하면 다음과 같은 `insert()` 함수를 사용할 수 있다.

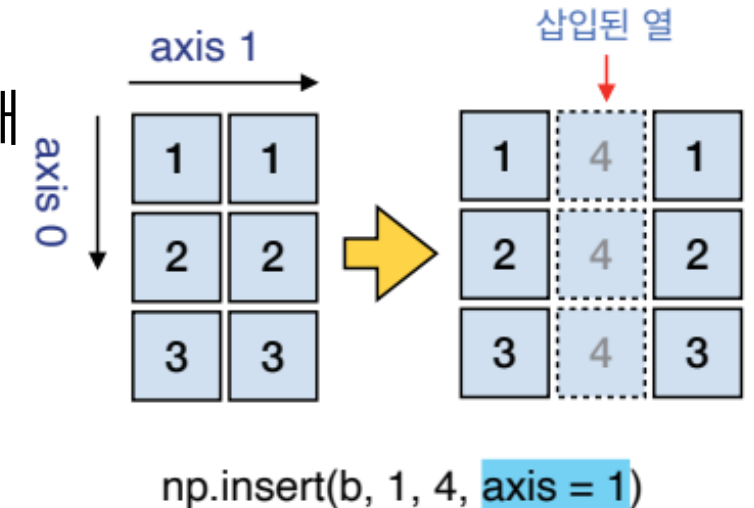
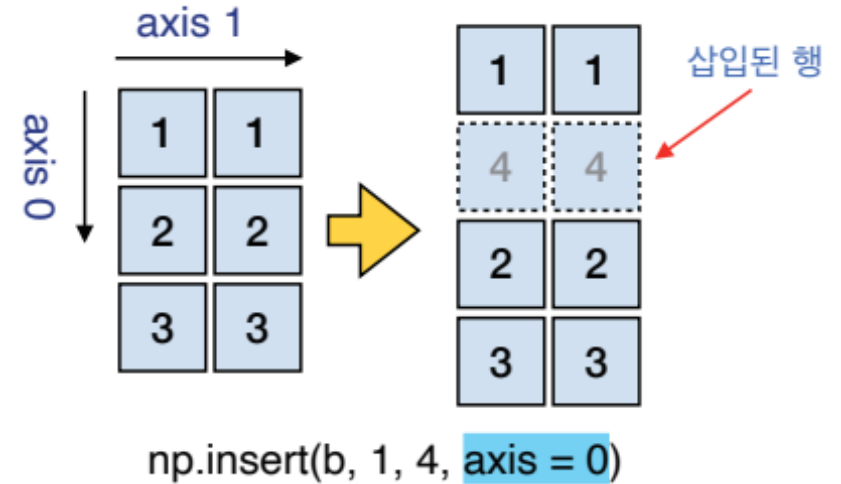


```
a = np.array([1, 3, 4])  
np.insert(a, 1, 2)
```

```
array([1, 2, 3, 4])
```

다차원 배열의 축과 삽입

- 이제 다음과 같은 2차원 배열에 대해서 특정 값을 행과 열에 삽입하는 경우를 고려해 보자.
- axis를 명시하여 `[[1, 1], [2, 2], [3, 3]]` 형상의 2차원 배열에 `insert(a, 1, 4, axis = 0)`를 하게 되면 `[[1, 1], [4, 4], [2, 2], [3, 3]]`와 같은 배열이 된다.
- `insert()` 함수의 첫 매개변수는 배열 객체이며, 두 번째는 삽입할 위치, 세 번째는 삽입할 값, 네 번째는 삽입 방향인데 `axis = 0`으로 할 경우 0축(그림의 axis 0) 방향으로 된다.



다차원 배열의 축과 삽입

- `insert(a, 1, 4, axis = 0)`는 원소 삽입 시에 4라는 스칼라값을 `[4, 4]`와 같은 벡터값으로 브로드캐스팅하므로, `insert(a, 1, (4, 4), axis = 0)`과 동일하다.
- `axis = 0`을 `axis = 1`로 변경할 경우 그림과 같이 축 1의 방향(2차원 배열에서 열 방향)의 두 번째 항목에 4를 삽입한다.
- 따라서, 원래의 `[1, 1]`을 `[1, 4, 1]`로, `[2, 2]`를 `[2, 4, 2]`로, `[3, 3]`을 `[3, 4, 3]`으로 모든 행의 값을 변경한다.



```
b = np.array([[1, 1], [2, 2], [3, 3]])  
np.insert(b, 1, 4, axis = 0)
```

```
array([[1, 1], [4, 4], [2, 2], [3, 3]])
```



```
np.insert(b, 1, 4, axis = 1)
```

```
array([[1, 4, 1],  
       [2, 4, 2],  
       [3, 4, 3]])
```

다차원 배열의 축과 삽입

- 또한, 넘파이에는 다음과 같은 `flip()` 함수가 있어서 지정된 축방향으로 원소들을 뒤집어 줄 수 있다.



```
c = np.array([[1, 2, 3], [4, 5, 6]])  
np.flip(c, axis=1)
```

```
array([[3, 2, 1],  
       [6, 5, 4]])
```



```
np.flip(c, axis=0)
```

```
array([[4, 5, 6],  
       [1, 2, 3]])
```

다차원 배열의 축과 삽입



배열에 원소 삽입하기

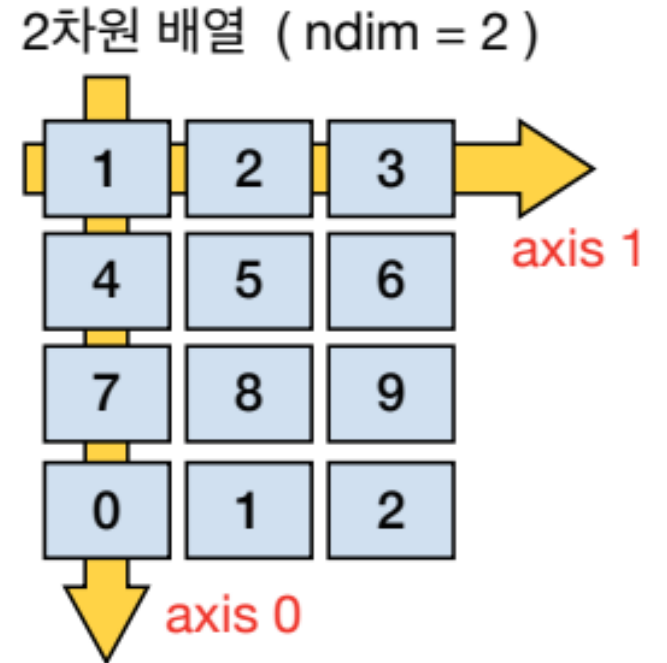
난이도 하

```
b = np.array([[1, 1], [2, 2], [3, 3]])
```

위의 b 배열에 `insert(b, 1, [1, 2, 3], axis=1)`을 수행하고 그 결과를 출력하여라.

넘파이 배열의 인덱싱과 슬라이싱

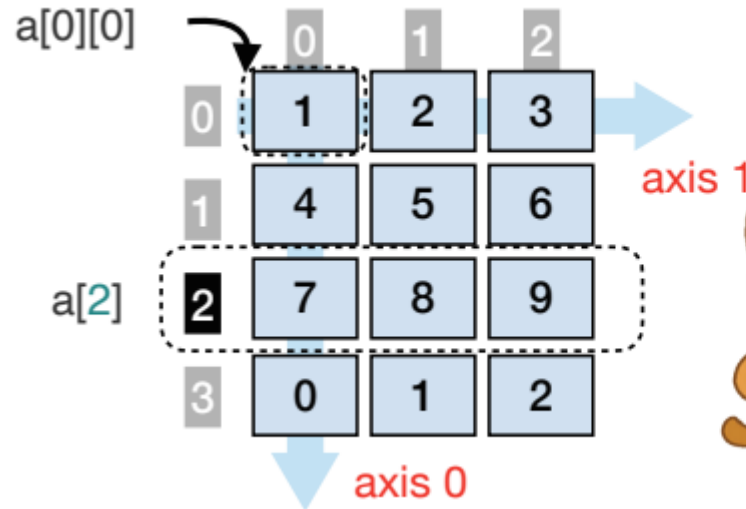
- 다차원 배열 내의 원소는 **인덱스**index라고 불리는 정수를 사용하여 참조할 수 있다.
- 리스트에서 사용했던 인덱싱과 슬라이싱은 넘파이 배열에도 거의 동일하게 적용된다.
- 1차원 배열의 인덱싱과 슬라이싱은 리스트와 큰 차이가 없으므로 2차원 배열을 이용하여 기본적인 인덱싱과 슬라이싱을 연습해 보자.
- 그림과 같은 2차원 배열을 만들어 보자. 이것은 아래와 같이 리스트의 리스트(이중 리스트)를 만들어 생성할 수 있다.



```
a = np.array( [[1, 2, 3], [4, 5, 6],  
               [7, 8, 9], [0, 1, 2]] )
```

넘파이 배열의 인덱싱과 슬라이싱

- 인덱싱을 먼저 수행해 보자. 인덱싱은 두 축에 대해서 모두 가능하다.
- 첫 번째 축으로 0, 두 번째 축으로 0을 지정하면 가장 첫 원소인 1이 나타날 것이다.
- 두 번째 축에 대해 인덱싱을 하지 않으면, 첫 번째 축에서 지정된 위치의 모든 데이터가 나타난다.



이차원 배열 `a`에서 `a[0][0]`은 1행 1열의 원소를 참조합니다. 한편 `a[2]`는 세번째 행을 참조합니다.



```
print(a[0][0])  
print(a[2])
```

1

[7, 8, 9]

넘파이 스타일의 슬라이싱과 논리 인덱싱

- 넘파이 스타일의 인덱싱을 이용하여 슬라이싱을 하면 각각의 슬라이싱 범위는 리스트의 슬라이싱처럼 순차적으로 적용되는 것이 아니라, 각각의 축에 별도로 적용된다.
- 따라서 다음과 같은 슬라이싱이 가능하다.

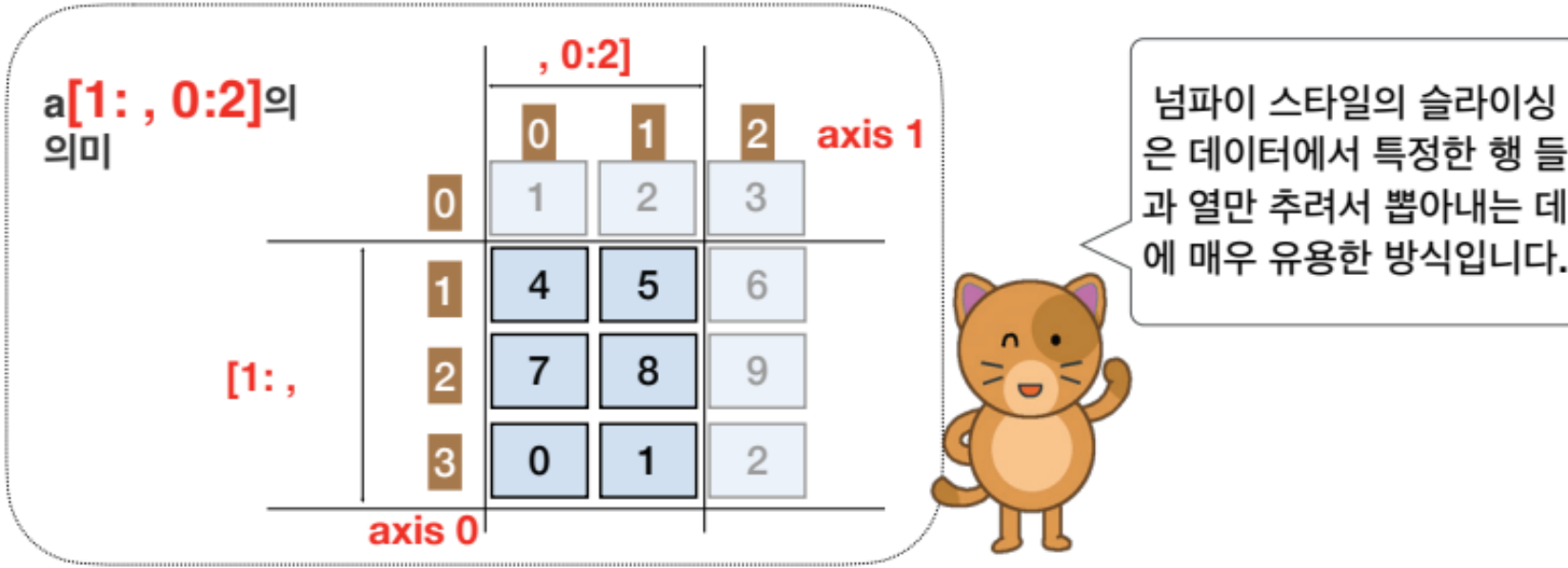


```
arr_2d = np.array([[1, 2, 3],  
                  [4, 5, 6],  
                  [7, 8, 9],  
                  [0, 1, 2]])
```

```
arr_2d[1: , 0:2]
```

```
array([[4, 5],  
       [7, 8],  
       [0, 1]])
```

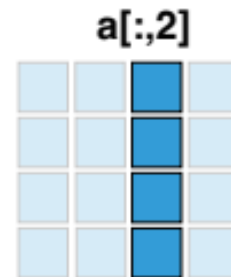
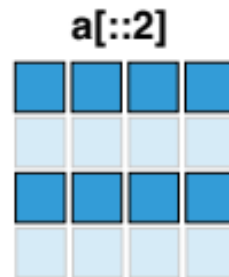
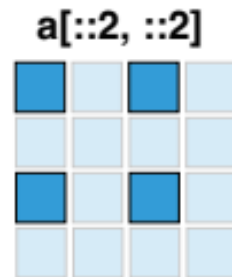
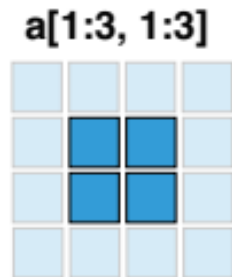
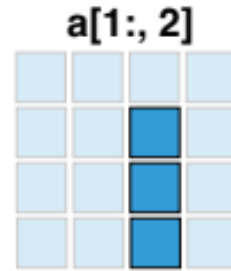
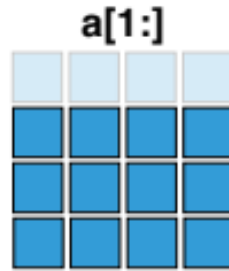
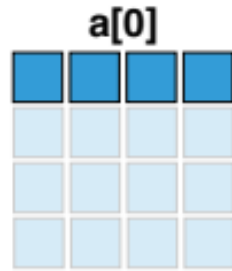
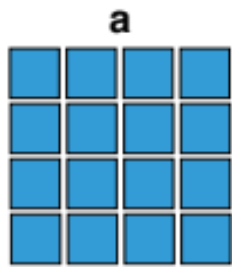
넘파이 스타일의 슬라이싱과 논리 인덱싱



- 이것은 행과 열로 이루어진 데이터에서 특정한 행들과 열만 추려서 뽑아내는 데에 매우 유용한 방식이며, 선형대수의 행렬 관련 기능을 구현하는 데에 매우 중요한 기능이기도 하다.
- 넘파이 스타일의 슬라이싱은 큰 행렬에서 작은 행렬을 끄집어내는 것으로 이해하면 된다.

넘파이 스타일의 슬라이싱과 논리 인덱싱

- 4x4 크기의 np_array라는 다차원배열의 인덱싱 및 슬라이싱 결과는 다음과 같다



심표와 콜론의 사용에 주의를 하셔야 합니다.



넘파이 스타일의 슬라이싱과 논리 인덱싱

- 넘파이는 데이터를 추려내는 데에 슬라이싱 뿐만 아니라 **논리 인덱싱** logical indexing이란 효율적인 방법도 제공한다. 이것은 특정한 어떤 조건을 주어서 배열에서 원하는 값을 추려내는 것이다.
- 넘파이 배열에 조건을 주면 불리언 배열이 만들어진다. 2차원 배열을 이용하여 확인해 보자.
- 아래의 예는 16개의 원소를 가진 2차원 넘파이 배열을 만들었다. 그리고, 동일한 크기의 배열인데, 각 원소가 "**2의 배수**"인 경우에 참이 되는 불리언 배열을 다음과 같이 만들 수 있다. 방법은 넘파이 배열을 조건식에 넣기만 하면 된다.



```
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12], [13,14,15,16]])  
a % 2 == 0
```

```
array([[False,  True, False,  True],  
       [False,  True, False,  True],  
       [False,  True, False,  True],  
       [False,  True, False,  True]])
```

넘파이 스타일의 슬라이싱과 논리 인덱싱

- 이렇게 얻어진 배열을 이용하여 인덱스로 사용하면 해당 조건에 부합하는 원소만 추출할 수 있다.
- 이렇게 하면 a 배열 내의 모든 원소 중에서 참인 위치의 값, 즉 2의 배수인 것만 추출된다.

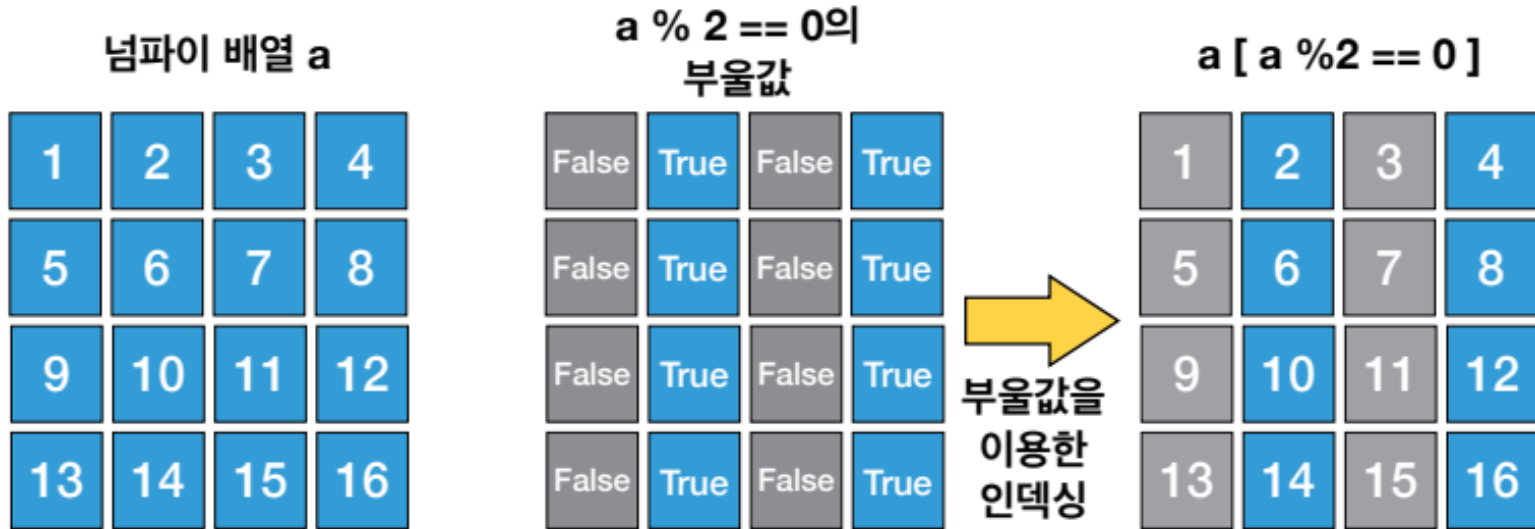


```
a[ a % 2 == 0 ]
```

```
array([ 2,  4,  6,  8, 10, 12, 14, 16])
```

넘파이 스타일의 슬라이싱과 논리 인덱싱

- 이 동작을 시각적으로 표시하면 다음과 같은 그림으로 설명할 수 있다.



- 넘파이의 다차원 배열은 데이터 과학자들이 다루는 벡터 형태의 데이터와 행렬을 다루기에 매우 적합한 기능들을 가지고 있는 것을 확인할 수 있었다.
- 넘파이는 다량의 수치 데이터를 고속으로 처리할 수 있는 데이터 병렬 처리 특성이 있다.

다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬

- 다차원 배열 ndarray의 메소드에 대하여 살펴보도록 하자.
- [10, 20, 30]의 세 원소를 가진 1차원 배열의 max(), min(), mean() 메소드는 최대값 30, 최소값 10, 평균값 20.0을 다음과 같이 출력한다.
- 만일 원소들의 자료형을 변환하고자 한다면 astype() 메소드를 사용한다.



```
a = np.array([10, 20, 30])  
a.max(), a.min()           # a 배열원소 중 최대값, 최소값을 출력함
```

```
(30, 10)
```



```
a.mean()                   # a의 평균을 출력함
```

```
20.0
```



```
a.astype(np.float64)       # 원소의 자료형을 float64로 변환
```

```
array([10., 20., 30.])
```

다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬

- 다차원 배열이 2차원 이상의 배열이라면 이를 1차원 배열로 변환할 필요성도 있는데 이때 사용하는 메소드가 바로 `flatten()` 메소드이다. 이 과정을 **평탄화**라고 한다.

```
b = np.array([[1, 1], [2, 2], [3, 3]])  
b.flatten()    # 넘파이 다차원 배열의 평탄화 메소드  
  
array([1, 1, 2, 2, 3, 3])
```

- 2차원 배열의 행과 열의 순서를 서로 교환하는 것을 **전치** `transpose`라고 하는데 다차원 배열의 전치는 `.T` 속성값을 이용하면 된다.

```
b = np.array([[1, 1], [2, 2], [3, 3]])  
b.T  
  
array([[1, 2, 3],  
       [1, 2, 3]])
```

다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬

- 파이썬의 리스트는 정렬을 위한 `sort()` 함수를 가지고 있는데 넘파이 다차원 배열 역시 정렬을 위한 `sort()` 함수를 가지고 있다.

```
 c = np.array([35, 24, 55, 69, 19, 99])  
c.sort()      # 배열의 정렬(오름차순)  
c[::-1]      # 내림차순 정렬  
  
array([99, 69, 55, 35, 24, 19])
```

- 2차원 배열이 있을 때 이 배열에 `sort()` 메소드를 사용하면 원소들은 `axis 1` 방향으로 오름차순 정렬이 된다.
- 키워드 인자로 `axis=0`을 넣어주면 `axis 0`의 방향으로 오름차순 정렬이 된다.

다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬



```
d = np.array([[35, 24, 55], [69, 19, 9], [4, 1, 11]])  
d.sort()      # 배열의 정렬, 디폴트 axis는 1  
d
```

```
array([[24, 35, 55],  
       [ 9, 19, 69],  
       [ 1,  4, 11]])
```



```
d = np.array([[35, 24, 55], [69, 19, 9], [4, 1, 11]])  
d.sort(axis=0)  # axis=0 방향 정렬  
d
```

```
array([[ 4,  1,  9],  
       [35, 19, 11],  
       [69, 24, 55]])
```

다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬



flip() 함수를 이용한 정렬 결과 보여주기

난이도 중

위의 결과를 수정하여 d 배열의 값이 axis 1, axis 0 방향에 대하여 각각 내림차순 정렬이 되도록 하고 그 결과를 출력하여라(힌트: 넘파이의 flip() 함수를 사용해보자).

```
array([[55, 35, 24],  
       [69, 19,  9],  
       [11,  4,  1]])
```

```
array([[69, 24, 55],  
       [35, 19, 11],  
       [ 4,  1,  9]])
```

다차원 배열을 위한 append() 함수

- append() 함수는 인자로 입력된 두 배열을 합하는 일을 하는데 axis를 통해서 축을 명시하지 않으면 디폴트 형상인 1차원 배열로 변환하게 된다.
- 이때 axis = 0과 같이 축을 명시해 준다면 다음과 같이 특정한 축으로 합하게 되어 배열의 차원이 유지된다.



```
a = np.array([1, 2, 3])  
b = np.array([[4, 5, 6], [7, 8, 9]])  
np.append(a, b)
```

```
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```



```
np.append([a], b, axis = 0)      # [a]를 통해 2차원 배열로 만들어야 함
```

```
array([[1, 2, 3],  
       [4, 5, 6],  
       [7, 8, 9]])
```

배열의 형태를 바꾸는 reshape()

- reshape() 메소드는 상당히 많이 사용되는 메소드이다. 데이터의 개수는 유지한 채로 배열의 차원과 형태를 변경한다. 이 메소드의 인자인 shape을 튜플의 형태로 넘겨주는 것이 원칙이지만 reshape(x, y)라고 하면 reshape((x, y))와 동일하게 처리된다.

변경하여 얻고 싶은 형태를 넘겨 줌
1차원: (n,)
2차원: (n, m)
3차원: (n, m, l)

new_array = old_array.reshape(shape)



이전 배열 old_array의 형태가 (n,m)이고, 새롭게 얻고 싶은 형태 shape이 (l,k)라고 하면 $n \times m = l \times k$ 를 만족해야만 형태를 바꿀 수 있습니다.

```
>>> y = np.arange(12)
>>> y
array([ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])
```

```
>>> y.reshape(3, 4)
array([[ 0, 1, 2, 3],
       [ 4, 5, 6, 7],
       [ 8, 9, 10, 11]])
```

배열의 형태가 3행 4열의 모양이 되도록 만듭니다

배열의 형태를 바꾸는 reshape() 과 flatten()

```
>>> y.reshape(6, -1)
array([[ 0,  1],
       [ 2,  3],
       [ 4,  5],
       [ 6,  7],
       [ 8,  9],
       [10, 11]])
```

인수로 -1을 전달하면 데이터의 개수에 맞춰서 자동으로 배열의 형태가 결정

```
>>> y.reshape(7, 2)
...
```

```
y.reshape(7, 2)
```

```
ValueError: cannot reshape array of size 12 into shape (7,2)
```

reshape()에 의해 생성될 배열의 형태가 호환되지 않을 경우 발생하는 오류

```
>>> y.flatten() # 2차원 배열을 1차원 배열로 만들어 준다
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

flatten()은 평탄화 함수로 2차원 이상의 고차원 배열을 1차원 배열로 만들어 준다.

다차원 배열을 위한 append() 함수



다차원 행렬의 생성과 활용

난이도 중

1에서 9까지의 정수 값을 크기 순서대로 가지는 3×3 크기의 행렬 `a`를 `arange()`와 `reshape()`을 이용하여 생성하여라. 모든 성분의 값이 3인 3×3 크기의 행렬 `b`를 `full()` 함수를 사용하여 생성하여라. `a`와 `b`를 다음과 같이 출력하여라.

```
a = [[1 2 3]
      [4 5 6]
      [7 8 9]]
```

```
b = [[3 3 3]
      [3 3 3]
      [3 3 3]]
```

과학자들이 사랑하는 수: 난수

- 데이터 과학을 위해서 사용하는 데이터는
 - 1) 실세계의 관측을 통해서 획득한 데이터,
 - 2) 실세계 데이터와 유사한 특성을 가지는 가상의 데이터를 사용하는 것이 일반적이다.
- 데이터 과학의 원래 목적을 생각해 본다면 당연히 실세계의 관측을 통해서 데이터를 획득하는 것이 바람직할 것으로 생각되지만, 이 경우 **많은 비용이 필요하며 장시간의 관측**이 필요한 경우가 많다.
- 따라서 실세계와 유사한 특성을 가지는 가상의 데이터를 만들어서 분석하고 시뮬레이션을 한 후에 이 모델을 실세계 데이터 분석에 적용하는 경우가 많다.
- 이를 위해서는 난수에 대한 개념을 이해하고 난수를 생성하는 알고리즘을 먼저 알아야 한다.



규칙성이 없는 수가 난수인
데요. 프로그램을 통해서
만든 규칙을 통해 난수를
생성할 수 있을까요?

과학자들이 사랑하는 수: 난수

- 난수란 **인간의 의도와 무관하게 생성된 수**를 의미하므로, 인간이 만든 기계에 특정한 알고리즘을 통해서 난수를 생성한다는 것 자체가 모순이다.
- 하지만 난수와 매우 유사한 특징을 가지는 수를 생성하는 것은 가능한데 이러한 난수는 **의사 난수** *pseudo random* 라고 한다.
- 의사(疑似)란 '의심스러운', '비슷한'을 의미하는데 진정한 의미의 난수가 아니지만, **난수와 유사한 기능을 한다는** 의미로 이 단어를 사용한다.
- 의사 난수 생성기의 특징은 이 생성기가 만들어낸 임의의 난수 값 (X_n) 을 다음 난수 값 (X_{n+1}) 을 만드는 재료로 사용한다는 점이다.
- 다음은 의사 난수 생성기의 예이다.

$$X_{n+1} = (aX_n + b) \bmod m$$

과학자들이 사랑하는 수: 난수

- 여기서 x 는 의사 랜덤 값의 **열sequence**이 되며, x_n 값은 x_{n+1} 값을 만드는 데 사용될 수 있다.
- a 값과 b , m 은 임의의 값으로 선택하면 되는데 여기서는 $a = 1103515245$, $b = 12345$, $m = 231$ 로 두고 의사 난수 함수를 만들어 보자.



```
def pseudo_rand(x):  
    a = 1103515245  
    b = 12345  
    m = 2 ** 31  
    new_x = (a * x + b) % m  
    return new_x
```

과학자들이 사랑하는 수: 난수

```
seed = 100          # 초기값을 임의로 설정하자
x = pseudo_rand(seed) # seed를 입력으로 하여 새로운 x를 만들자
print(x)
x = pseudo_rand(x)   # x_n을 입력으로 하여 새로운 x_(n+1)을 만들자
print(x)
```

```
829870797
1533044610
```

- 난수를 생성하는 코드에는 다음 난수를 생성하기 위한 최초 값이 필요한데 이를 **시드값** seed number이라고 한다.
- 보통 시드값은 컴퓨터로부터 시간 정보를 얻어서 이 값을 사용하는 경우가 많다.
- 이 책에서 사용하게 될 난수 값은 모두 **의사 난수로 생성한 값**이지만 이를 구분하지 않고 **난수 값으로 지칭**하도록 한다.

과학자들이 사랑하는 수: 난수

- 난수를 생성하는 함수나 모듈은 대부분의 프로그래밍 언어에서 라이브러리 형태로 제공한다.
- 넘파이로 가상의 성인 10명의 키를 생성하는 경우를 고려해 보자.
- 이 데이터가 150에서 190센티미터 사이의 임의의 값을 가진다고 가정하자.
- 이 값이 정수값일 경우 넘파이의 random서브 모듈에 있는 randint() 함수를 사용할 수 있다.
- randint() 함수는 인자로 최소값, 최대값, 원소의 개수를 줄 수 있다.
- 이때, 최대값이 191일 경우 191은 원소에 포함되지 않는다.



```
np.random.randint(150, 191, size=10)
```

```
array([186, 157, 167, 153, 159, 173, 176, 187, 163, 179])
```

다양한 난수 만들기 함수를 살펴보자

- 넘파이는 풍부한 난수 생성 기능을 제공하므로 이것을 활용하면 데이터 과학을 위한 임의의 데이터를 쉽게 얻을 수 있다. 넘파이의 난수는 random 서브 모듈에서 제공하는데 randn() 함수는 표준 정규 분포를 따르는 난수를 생성하는 데 사용된다.
- 만일 이 데이터들이 평균값이 165, 표준편차가 10인 정규 분포를 따른다고 할 경우 다음과 같은 방법을 사용할 수 있다.
- randn(5)는 5개의 난수를 생성하는데 이 난수는 평균이 0이고 분산이 1인 **표준 정규 분포** standard normal distribution를 따른다. 따라서 이 값에 10을 곱하고 165를 더하게 되면 우리가 원하는 데이터를 얻을 수 있다.



```
rnd = np.random.randn(5) * 10 + 165 # 평균값이 165, 표준편차가 10인 값 생성  
rnd
```

```
array([176.07979332, 177.91821264, 165.45715947, 164.34633461,  
       167.9482772 ])
```

다양한 난수 만들기 함수를 살펴보자

- 이 결과에 `round(2)`와 같은 방법으로 소수점 아래 숫자의 범위를 제한하거나 `astype(int)` 메소드를 사용하여 정수형 자료로 변환할 수 있다.

```
▶ rnd.round(2) # 소수점 아래 둘째 자리까지 값으로 변환
```

```
array([176.08, 177.92, 165.46, 164.35, 167.95])
```

```
▶ rnd.astype(int) # 정수형 자료형으로 변환
```

```
array([176, 177, 165, 164, 167])
```

- 만일 `seed`를 다음과 같이 42와 같은 특정한 값으로 지정하면 매번 동일한 난수의 열을 얻을 수 있다.

```
np.random.seed(42) # 시드값이 특정되면 매번 동일한 난수의 열이 생성됨
```

다양한 난수 만들기 함수를 살펴보자

- 정규 분포는 매우 중요한 난수로 `normal()` 함수를 사용해서 얻을 수도 있다.
- 이전에 살펴본 평균값이 165, 표준편차가 10인 정규 분포 함수는 다음과 같이 생성 가능하다.
- Location (`loc`) : 중심위치



```
nums = np.random.normal(loc=165, scale=10, size=(3, 4)).round(2)
nums
```

```
array([[160.32, 156.77, 164.35, 157.87],
       [174.06, 172.66, 173.26, 151.76],
       [147.48, 175.02, 170.45, 183.95]])
```

다양한 난수 만들기 함수를 살펴보자

- 다음은 random 서브 모듈에 포함된 중요한 난수 관련 함수이다.

함수 이름	설명
rand()	균등분포의 난수 표본을 추출한다.
randn()	표준정규분포의 난수 표본을 추출한다.
randint()	최소값과 최대값을 인자로 받아, 주어진 범위 내의 정수 난수를 생성한다. 이때 최대값은 포함하지 않는다.
normal()	평균과 표준 편차를 인자로 받아, 정규 분포의 난수 표본을 추출한다.
uniform()	균등 분포에서 표본을 추출한다.
shuffle()	주어진 리스트나 다차원 배열의 순서를 임의로 섞는다.
permutation()	주어진 범위의 수를 생성하여 섞거나, 배열을 임의의 순서로 섞는다.

다양한 난수 만들기 함수를 살펴보자

- 다음은 이 함수들을 사용하여 임의의 수를 생성하거나 섞는 예시이다.
- shuffle: 원본 배열이 직접 섞여 변경됨
- Permutation : 섞인 새로운 배열을 반환



```
a = np.arange(10)      # 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 값을 생성
np.random.shuffle(a)
a                       # a에는 순서가 뒤섞인 값이 들어있다
```

```
array([2, 8, 9, 6, 4, 0, 1, 5, 7, 3])
```



```
np.random.permutation([2, 4, 6, 8, 10])
```

```
array([ 8,  4, 10,  6,  2])
```

다양한 난수 만들기 함수를 살펴보자



다차원 배열을 나누어 보자

난이도 중

`arange()`를 사용하여 1에서 50까지의 원소를 가지는 다차원 배열을 만들자. 이 배열의 원소 50개를 랜덤하게 섞은 후 80%의 데이터는 `train_data`에 넣고 나머지 20%의 데이터는 `test_data`라는 배열에 넣어서 이 두 개의 배열을 반환하는 함수 `train_test_split()`을 만들자. 반환된 배열 값을 각각 출력하여라.

평균, 분산, 표준 편차를 알아보자

- 통계학에서 **평균**average은 주어진 데이터 집합의 중심값 또는 대표값을 나타내는 척도
- 산술 평균 : 모든 데이터 값의 합을 데이터의 개수로 나눈 값
 - 데이터 집합의 일반적인 값을 나타낸다.



```
a = np.array([10, 20, 30, 40, 50])  
print('a의 평균값 :', a.mean())
```

```
a의 평균값 : 30.0
```

평균, 분산, 표준 편차를 알아보자

- **분산**variance은 데이터가 평균으로부터 얼마나 퍼져 있는지를 나타내는 척도
 - 분산이 크면 데이터가 평균값으로부터 멀리 퍼져 있음을, 분산이 작으면 데이터가 평균값에 가깝게 모여 있음을 의미한다
- A 모둠 : 5명의 학생이 시험에서 받은 점수가 각각 60, 70, 80, 90, 100점이라고 가정
- B 모둠 : 점수가 각각 76, 78, 80, 82, 84점이라고 가정하자
 - A 모둠의 평균 : $(70 + 80 + 80 + 90 + 100) / 5 = 80$
 - B 모둠의 평균 : $(76 + 78 + 80 + 82 + 84) / 5 = 80$
- 평균이 같다고 해서 데이터의 성질들이 같다고 볼 수는 없음
 - A 모둠 5명은 매우 점수가 높은 학생과 매우 점수가 낮은 학생들이 모여 있으며, B 모둠은 비교적 점수 차이가 적은 학생들이 모여 있다.
 - 이러한 성질을 정량화 하는 방법이 분산이다.

평균, 분산, 표준 편차를 알아보자

- A 모둠 : 점수가 각각 60, 70, 80, 90, 100점이라고 가정
- B 모둠 : 점수가 각각 76, 78, 80, 82, 84점이라고 가정
 - A 모둠의 평균 : $(60 + 70 + 80 + 90 + 100) / 5 = 80$
 - B 모둠의 평균 : $(76 + 78 + 80 + 82 + 84) / 5 = 80$



두 모둠 학생들의 성적 평균은 모두 80점입니다. 하지만 B 모둠 학생들의 성적 분포가 A 모둠에 비해 일정한 것을 볼 수 있지요.

평균, 분산, 표준 편차를 알아보자

- A 모둠의 분산을 구하는 방법은 각 점수와 평균과의 차이(편차)를 계산하고, 이를 다음과 같이 제곱한 다음 이들의 평균을 구하면 된다.
 - 편차의 제곱을 취하는 이유는 편차값이 음수 또는 양수가 될 수 있기 때문에 이들의 합을 구할 경우 편차가 크더라도 이들의 합은 0에 가까운 수가 되기 때문이다. 우선 각 점수에 대한 편차의 제곱을 구하자.
 - 60점의 편차 제곱: $(60 - 80)^2 = 400$
 - 70점의 편차 제곱: $(70 - 80)^2 = 100$
 - 80점의 편차 제곱: $(80 - 80)^2 = 0$
 - 90점의 편차 제곱: $(90 - 80)^2 = 100$
 - 100점의 편차 제곱: $(100 - 80)^2 = 400$



$$(400 + 100 + 0 + 100 + 400) / 5 = 1000 / 5 = 200$$

평균, 분산, 표준 편차를 알아보자

- A 모둠 : 점수가 각각 60, 70, 80, 90, 100점
- B 모둠 : 점수가 각각 76, 78, 80, 82, 84점



```
A = np.array([60, 70, 80, 90, 100])  
print('A 모둠의 분산:', np.var(A))
```

A 모둠의 분산: 200.0



```
B = np.array([76, 78, 80, 82, 84])  
print('B 모둠의 분산:', np.var(B))
```

B 모둠의 분산: 8.0

- 분산을 통해서 데이터가 평균값 주변에 모여있는 정도를 측정할 수 있으나, 이 값은 편차의 제곱값들 합의 평균값이다.
 - 이 편차의 제곱은 실제 값과 너무 동떨어진 값이 될 수 있기 때문에 이를 실제값과 근사시키기 위하여 분산에 제곱근을 씌워서 사용하는 것이 더 합당할 것이다.
- 이 값이 바로 **표준 편차** standard deviation이다.
- 표준 편차를 구하는 넘파이 함수는 `std()`이다.

평균, 분산, 표준 편차를 알아보자

- m 개의 인스턴스를 가진 데이터 x_i 의 평균은 그리스 문자 μ_x 로 표기하며, 분산은 σ_x^2 으로, 표준 편차는 그리스 문자 σ_x 표기를 많이 사용한다.

심화 넘파이는 왜 성공했나

- 넘파이는 다량의 수치 데이터를 다루고, 일부를 추출하는 데에 효율적인 기능을 제공하고 있지만, 이것만으로 넘파이의 성공 이유를 모두 설명할 수는 없다.
- 넘파이는 사실 강력한 **배열 프로그래밍** `array programming` 기능 덕분에 오늘의 인기를 누리게 되었다.
- 배열 프로그래밍은 데이터에 연산을 실시할 때, 데이터를 구성하는 값 전체에 대해 한 번에 연산이 적용되도록 하는 방식을 의미한다.
- 이러한 해법을 제공하지 못하는 언어를 **스칼라** `scalar` 언어라고 하며, 전통적인 C, C++ 등은 스칼라 언어에 해당한다.

심화 넘파이는 왜 성공했나

- 배열 프로그래밍은 데이터를 다루는 연산의 표현을 간결하게 만든다. 예를 들어 두 개의 배열 `arr_a`와 `arr_b`가 있고, 두 배열 원소의 개수는 `arr_len`이라고 가정해 보자. 스칼라 언어를 사용한다면 이 두 배열을 더하는 코드는 다음과 같은 의사 코드로 표현될 것이다.

```
For i from 0 to arr_len-1
  arr_sum[i] = arr_a[i] + arr_b[i]
end For
```

- 그러나 배열 프로그래밍을 지원하는 언어는 다음과 같은 표현을 지원한다.

```
arr_sum = arr_a + arr_b
```

심화 넘파이는 왜 성공했나

- 위와 같은 표현은 연산의 대상이 되는 **피연산자**operand가 다수의 원소를 가진 배열, 즉 수학적으로 여러 수로 이루어진 벡터 데이터라는 것을 가정하기 때문에 가능하다.
- 여기서 더하기 연산자는 배열과 배열, 수학적 표현으로는 벡터와 벡터의 더하기를 의미하는 것이다.
- 이러한 이유로 배열 프로그래밍에서 사용하는 연산을 **벡터화 연산**vectorized operation이라고 한다.

심화 넘파이는 왜 성공했나

- 물론 표현의 간결성만이 배열 프로그래밍의 장점이 아니다. 배열 프로그래밍이 사용하는 벡터화 연산은 강력한 병렬 처리가 뒷받침될 때 완성된다.
- 즉, **For 반복문을 사용한 스칼라 언어가 각각의 원소들을 순차적으로 처리해야** 한다면, 벡터화 연산은 두 배열의 대응하는 원소를 서로 더해서 새로운 배열에 저장하는 일을 차례를 기다리지 않고 동시에 수행할 수 있다.
- 이렇게 데이터를 잘게 쪼개어 각각에 대해 동일한 연산을 독립적으로 수행하는 방식을 **데이터 병렬성** data parallelism이라고 한다.

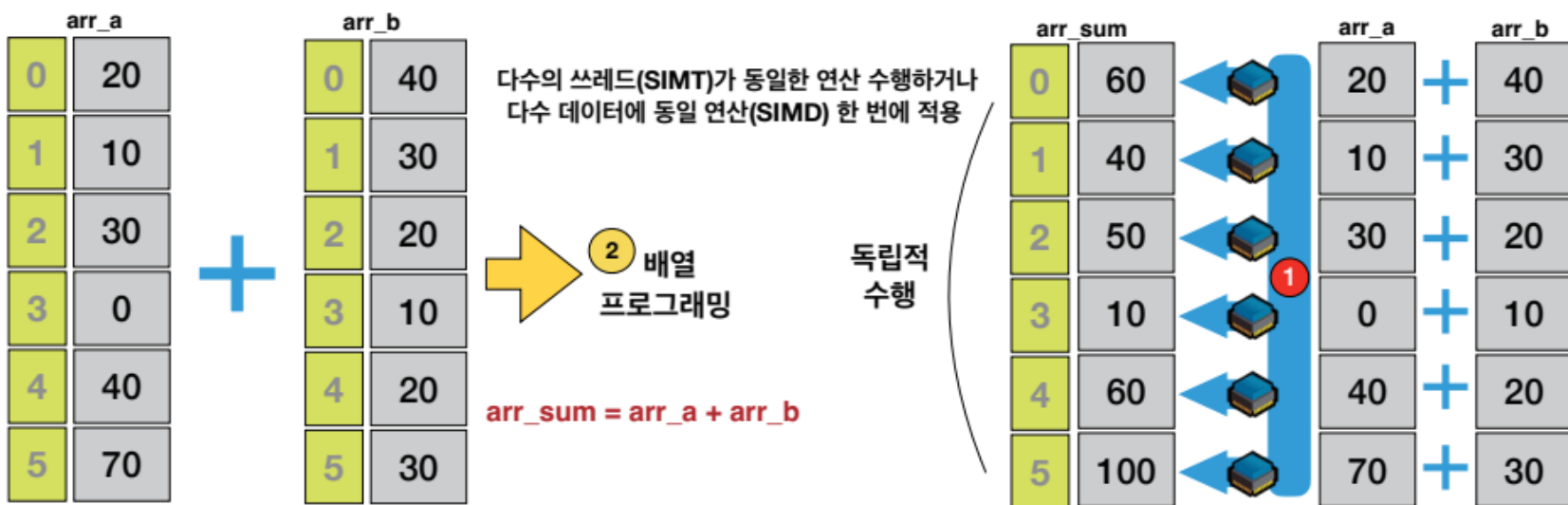
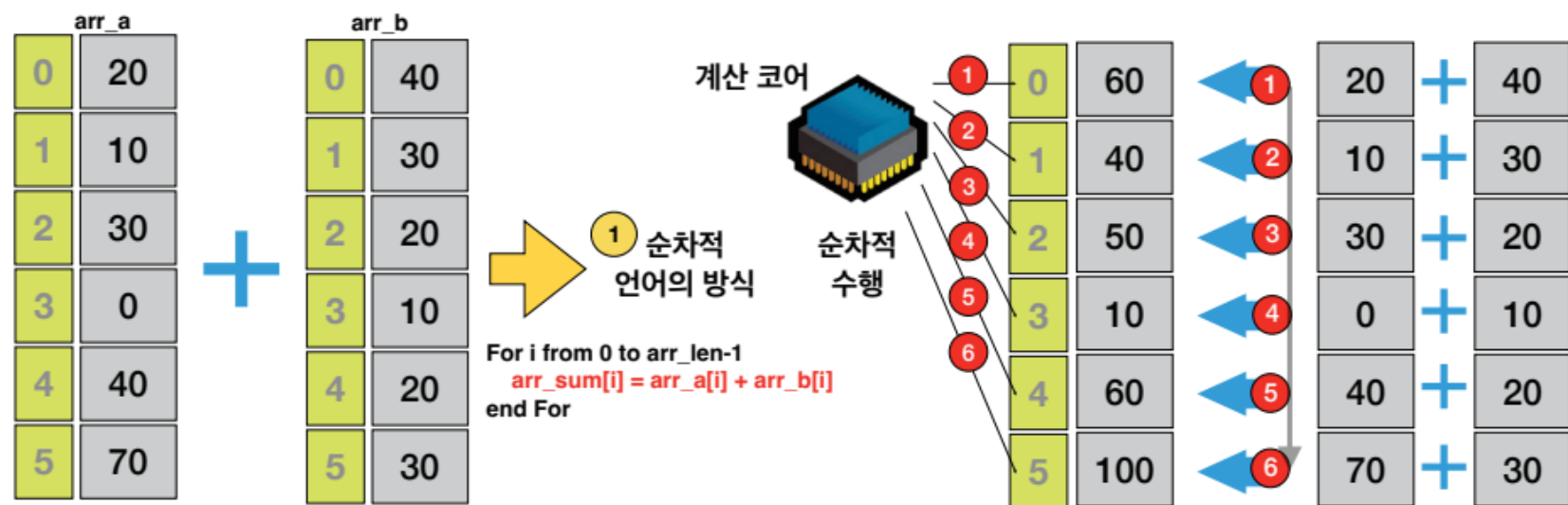
심화 넘파이는 왜 성공했나

- 프로그래밍에서 문제를 해결할 때 사용하는 병렬성은 크게 두 가지 유형으로 구분된다.
- 하나는 전통적 병렬 프로그래밍의 주제인 **작업 병렬성** task parallelism이다. 이것은 문제를 해결하는 방법을 이루는 작은 작업들 사이의 의존성을 분석한 뒤 동시에 처리할 수 있는 일은 여러 개의 계산 코어가 동시에 진행하는 방식이다.
- 또 다른 방식은 처리해야 하는 데이터를 잘게 쪼개어 각각의 조각에 대해 동일한 작업을 여러 개의 코어가 동시에 진행하는 방식이다. 이것을 데이터 병렬성이라고 부른다.
- 이 방식은 엄청나게 많은 픽셀을 그려야 하지만, 각각의 픽셀을 그리는 데에 필요한 작업은 동일한 절차를 따르면 되는 컴퓨터 그래픽스와 같은 분야에 적합하다.
- 이러한 이유로 **그래픽 처리 장치** GPU는 고성능 데이터 병렬 처리기로 발전하여 현재 슈퍼컴퓨팅 자원으로 적극 활용되고 있는 것이다.

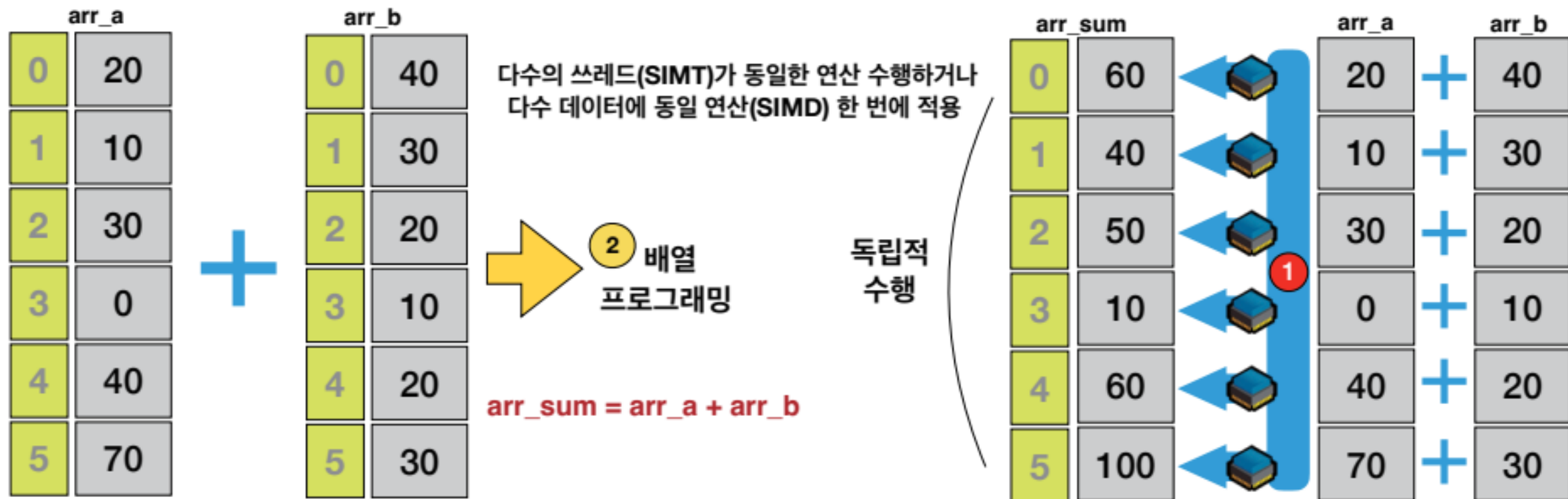
심화 넘파이는 왜 성공했나

- GPU는 다수의 **쓰레드** thread를 만들어 동일한 연산을 수행하게 하는데, 이러한 구조를 **단일 명령어 다중 쓰레드** single instruction multiple thread:SIMT 방식이라고 한다.
- CPU는 GPU와 달리 쓰레드를 생성하지 않고, 하나의 연산이 다수의 데이터에 동시에 적용되도록 하는 **단일 명령어 다중 데이터** single instruction multiple data:SIMD 방식을 사용한다.
- 전통적인 언어로도 이러한 데이터 병렬성을 활용할 수 있지만, 벡터화 연산을 그대로 표현하는 효율적인 코딩은 어렵다.
- 넘파이는 강력하고 압축적인 프로그래밍 구문을 제공하며, 프로그래머가 효율적으로 고성능의 데이터 병렬 처리를 할 수 있게 해준다.
- 넘파이의 성공 이유로 꼽아야 할 가장 중요한 요소가 바로 이것이다.

심화 넘파이는 왜 성공했나



심화 넘파이는 왜 성공했나



벡터화 연산의 성능을 측정해 보자

- 벡터화 연산은 하나의 명령이 여러 데이터에 동시에 적용된 방식으로 **배열 프로그래밍** `array programming`의 핵심 요소이다.
- 큰 작업을 분할하여 다수의 프로세서에서 나누어 처리하는 전통적 병렬 처리와 달리 하나의 프로세서 내에서 병렬적 데이터 처리가 묵시적으로 이루어진다.
- 현재 대다수의 CPU가 SIMD 방식을 지원하고 있는데 넘파이는 따로 설정하지 않아도 이를 활용한다.

벡터화 연산의 성능을 측정해 보자

- 넘파이의 배열 프로그래밍 성능 측정을 위해 time 모듈도 함께 임포트한다.
- 그리고 100개의 원소를 가진 두 개의 배열 arr_a와 arr_b를 만들고 난수를 채워 보자.
- 그리고 이 두 배열을 연산하여 그 결과를 담은 배열 result를 만들고, 여기에는 100개의 0을 채워 보자.



```
import numpy as np
import time          # 수행시간 측정을 위하여 time 모듈을 가져온다

arr_a = np.random.rand(100)
arr_b = np.random.rand(100)
result = np.zeros(100)
```

벡터화 연산의 성능을 측정해 보자

- 두 배열을 원소별로 곱하는 일은 전통적인 for 반복문을 이용하여 다음과 같이 할 수 있을 것이다.
- 이때 작업을 시작하기 전의 시간과 작업 종료 이후의 시간을 기록하여 차이를 구하면 계산에 소요된 시간을 알 수 있다.



```
start = time.time()
for i in range(len(arr_a)):
    result[i] = arr_a[i] * arr_b[i]
end = time.time()
print('소요시간: ', end - start)
```

명시적인 작업 지시 : C 스타일

소요시간: 0.0012476444244384766

벡터화 연산의 성능을 측정해 보자

- 100개의 원소를 가진 두 배열을 원소별로 곱하여 result에 담는 데에는 물론 오랜 시간이 걸리지 않고, 아무런 오류 없이 잘 동작하지만, 벡터화 연산을 사용하여 다시 써보자.



```
start = time.time()
result = arr_a * arr_b          # 벡터화 연산 *
end = time.time()
print('소요시간: ', end - start)
```

```
소요시간: 0.0000588893890380859
```

벡터화 연산의 성능을 측정해 보자

- 계산 시간은 대략 20배 정도 단축되었다.
- 이때 연산자 *는 벡터화 연산자로 배열의 모든 원소에 대해 한꺼번에 적용된다.
- 이 연산자뿐만 아니라 사칙 연산 모두와 나중에 다룰 행렬곱 연산 등이 모두 벡터화 연산으로 동작한다.
- 배열의 크기를 100,000개로 늘려 보자.



```
n_elem = 100000      # 원소의 개수  
arr_a = np.random.rand(n_elem)  
arr_b = np.random.rand(n_elem)  
result = np.zeros(n_elem)
```

벡터화 연산의 성능을 측정해 보자

- 아래와 같이 100,000개의 원소를 가진 배열에 같은 연산을 하고 비교해 보자. 연산을 수행하는 데에 걸리는 시간은 60배 정도 차이가 나는 것을 알 수 있다.



```
start = time.time()
for i in range(n_elem):           # 명시적인 작업 지시 : C 스타일
    result[i] = arr_a[i] * arr_b[i]
end = time.time()
print('원소별 접근 방식 소요시간: ', end - start)

start = time.time()
result = arr_a * arr_b
end = time.time()
print('벡터화 연산 수행 소요시간: ', end - start)
```

```
원소별 접근 방식 소요시간: 0.05570673942565918
벡터화 연산 수행 소요시간: 0.0009524822235107422
```

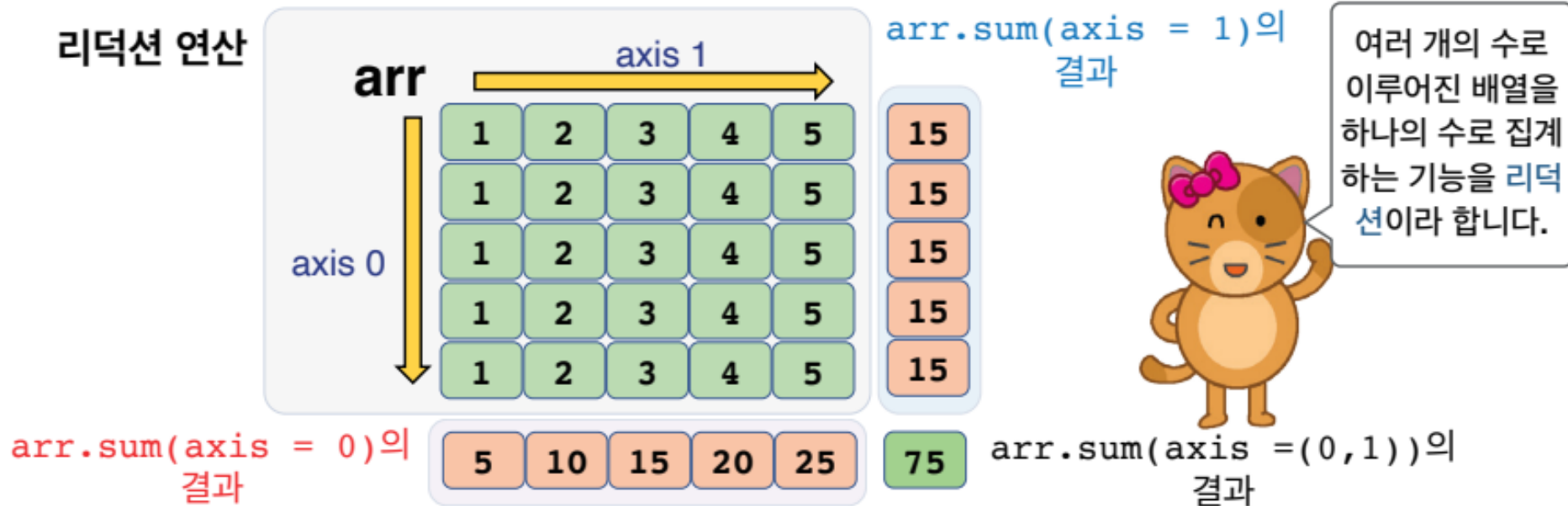
벡터화 연산의 성능을 측정해 보자

- 1차원 배열만 예를 들었지만, 2차원 이상의 고차원 배열에 대해서도 동일하게 원소별로 연산이 SIMD 방식으로 적용된다.
- 위와 같이 1차원 배열에서 원소별 접근을 했을 때, 벡터화 연산을 수행한 것에 비해 몇 배의 시간이 걸리는지를 성능 값으로 제시하여 비교한 결과가 다음과 같다.
- K는 1,000개, M은 1,000,000개를 의미한다.
- 일정한 규모 이상에서 벡터화 연산을 사용한 것이 **100배에서 200배까지 빠른 것**을 확인할 수 있다.

N	10K	20K	40K	100K	200K	400K	1M	2M	3M	4M	5M
성능	46	148	126	143	150	179	187	195	177	182	206

리덕션 : 배열을 더 강력하게 만드는 기능

- 넘파이의 다차원 배열은 **리덕션**reduction이라는 이름의 멋진 기능을 제공한다.
- 리덕션이라는 것은 여러 개의 수로 이루어진 배열을 하나의 수로 **집계**aggregation하는 것이다.
- 넘파이는 이 집계를 위한 함수로 `sum()`, `mean()`, `min()`, `max()` 같은 함수를 제공한다.



리덕션 : 배열을 더 강력하게 만드는 기능

- 실제 코드를 통해 이 연산을 수행해 보자. 위의 그림과 같은 결과가 나온 것을 확인할 수 있다.



```
arr = np.array([[1,2,3,4,5],
                [1,2,3,4,5],
                [1,2,3,4,5],
                [1,2,3,4,5],
                [1,2,3,4,5],
                ])
print(np.sum(arr, axis = 0))
print(np.sum(arr, axis = 1))
print(np.sum(arr, axis=(0, 1))) # axis = (0, 1)은 생략 가능
```

```
[ 5 10 15 20 25]
[15 15 15 15 15]
75
```

리덕션 : 배열을 더 강력하게 만드는 기능

- 리덕션 연산은 벡터화 연산을 통해 이루어지므로 높은 성능을 보여 준다.
- 큰 규모의 배열을 만들고 각 원소의 합을 원소별 접근을 통해서 계산하는 방법과 리덕션 기법을 이용한 방법으로 수행해 보고, 소요 시간을 비교해 보자.



```
import numpy as np
import time

n = 1000
arr = np.random.rand(n, n)
```

리덕션 : 배열을 더 강력하게 만드는 기능

- 이 배열은 총 1,000,000개의 원소를 가진다. 배열 원소의 합은 모든 원소에 접근해 그 값을 변수 sum_naive에 누적하면 된다.
- 결과와 소요 시간을 출력해 보자. 구글 코랩에서 0.4초 정도가 걸렸다.



```
sum_naive = 0
start = time.time()
for i in range(n):
    for j in range(n):
        sum_naive += arr[i,j]
end = time.time()
print(n, 'x', n, "배열의 원소 합", sum_naive, "\n계산 시간: ", end-start)
```

```
1000 × 1000 배열의 원소 합 500048.24990126066
계산 시간: 0.39148378372192383
```

리덕션 : 배열을 더 강력하게 만드는 기능

- 이것을 벡터화 연산이 적용되는 리덕션으로 구해보자. 아래와 같이 구현할 수 있다.
- 원소의 합은 동일한 값이 출력되고, 소요 시간은 0.0013초로 **300배 이상 빨라진** 것을 확인할 수 있다.



```
start = time.time()
sum_reduction = np.sum(arr, axis = (0,1))
end = time.time()
print(n, 'x', n, "배열의 원소 합", sum_reduction, "\n계산 시간: ", end-start)
```

```
1000 × 1000 배열의 원소 합 500048.2499012628
계산 시간: 0.00128173828125
```

선형방정식을 풀어보자

- 넘파이에는 `numpy.linalg`이라는 서브 모듈이 있으며 이 모듈은 선형대수를 위한 다양한 함수를 제공한다.
- 아래와 같이 x, y 두 개의 미지수를 가지는 선형 연립 방정식이 있다고 가정하자.

$$x + 2y = 6$$

$$x - 3y = 1$$

- 이 방정식의 해를 넘파이의 `linalg` 서브 모듈에서 다음과 같은 함수를 호출하여 구할 수 있다.



```
a = np.array([[1, 2], [1, -3]])  
b = np.array([6, 1])  
s = np.linalg.solve(a, b)  
print(s)
```

```
[4. 1.]
```

선형방정식을 풀어보자

- 연립 방정식이 해를 가지는 경우도 있지만 그렇지 않은 경우도 있다. 이를 판별하기 위해서는 행렬식을 이용할 수 있다.
- **행렬식**determinant은 정방 행렬에 수를 대응시키는 함수인데, 연립 방정식이 해를 가지는지 아닌지를 판별하는 데 사용할 수 있다.
- 다음은 2×2 , 3×3 크기의 행렬에 대한 행렬식을 구하는 방법이다.

- 2×2 행렬의 행렬식 $\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$

- 3×3 행렬의 행렬식 $\det \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = aei + bgf + cdh - ceg - bdi - afh$

선형방정식을 풀어보자

- 2×2 행렬과 3×3 행렬의 행렬식은 위의 수식을 이용하여 구할 수 있는데, 넘파이를 이용하면 아래와 같이 `det()`라는 한 줄의 함수 호출을 통해 간단하게 구할 수 있다.



```
a = np.array([[2, 1], [4, 5]])  
det_a = np.linalg.det(a)           # a 행렬의 행렬식  
b = np.array([[1, 2], [3, -6]])  
det_b = np.linalg.det(b)           # b 행렬의 행렬식  
print(det_a, det_b)
```

```
6.0 -12.0
```

선형방정식을 풀어보자



다차원 행렬의 생성과 활용(난이도 : 중)

1. 다음과 같은 연립방정식의 해를 numpy.linalg의 solve() 함수를 사용하여 구하여라.

$$\begin{aligned}x + y - z &= 0 \\2x - y + 3z &= 9 \\x + 2y + z &= 8\end{aligned}$$

2. 1번 문제의 연립방정식은 다음과 같은 행렬 A를 가진다.

```
A = np.array([[1, 1, -1], [2, -1, 3], [1, 2, 1]], dtype = 'int32')
```

이 행렬의 행렬식을 linalg.det() 함수를 사용하여 다음과 같이 구하여라.

```
det(A) = -11.0
```

배열의 결합 concatenate, vstack, hstack

- 여러 개의 다차원 배열을 결합하여 하나의 다차원 배열을 만들 때 사용할 수 있는 넘파이 함수에 대해서 알아보자. 우선 다음과 같은 다차원 배열 a, b, c와 리스트 d를 만들어 보자.



```
a = np.arange(10, 18).reshape(2,4)
```

a # (2, 4) 형상의 다차원 배열로 10에서 17까지의 원소를 가짐

```
array([[10, 11, 12, 13],  
       [14, 15, 16, 17]])
```



```
b = np.arange(12).reshape(3,4)
```

b # (3, 4) 형상의 다차원 배열로 0에서 11까지의 원소를 가짐

```
array([[ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11]])
```

배열의 결합 concatenate, vstack, hstack

```
▶ c = np.arange(8).reshape(2,4)  
c # (2, 4) 형상의 다차원 배열로 0에서 7까지의 원소를 가짐
```

```
array([[0, 1, 2, 3],  
       [4, 5, 6, 7]])
```

```
▶ d = list(range(4))  
d # d는 1차원 리스트로 0에서 3까지의 원소를 가짐
```

```
[0, 1, 2, 3]
```

- 리스트를 결합하기 위해서는 + 연산자를 사용할 수 있으나 다차원 배열의 +는 두 배열의 원소의 합이 되므로 이 연산자 대신 다음과 같이 넘파이의 concatenate() 함수를 사용해서 두 리스트를 결합할 수 있다.

```
▶ np.concatenate((a, b)) # a 배열의 마지막 원소 다음에 b 배열을 넣는다
```

```
array([[10, 11, 12, 13],  
       [14, 15, 16, 17],  
       [ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11]])
```

배열의 결합 concatenate, vstack, hstack

- 이 concatenate() 함수는 a 배열의 마지막 원소 다음에 b 배열의 원소를 차례로 넣는 방식으로 두 다차원 배열을 결합하고 이를 반환한다.
- 넘파이의 vstack(), hstack()은 이와 비슷하게 수직 방향(vertical)과 수평 방향(horizontal)으로 다차원 배열을 결합한다.
- 위의 함수는 다음과 동일한 함수로 구현할 수 있다.



```
np.vstack((a, b)) # a 배열과 b 배열을 수직 방향으로 결합
```

```
array([[10, 11, 12, 13],  
       [14, 15, 16, 17],  
       [ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11]])
```

배열의 결합 concatenate, vstack, hstack

- 넘파이의 vstack()은 배열과 리스트를 결합할 수 있으며, 다차원 배열 3개를 한꺼번에 결합하는 것도 가능하다.



np.vstack((a, d)) # a 배열과 d 리스트의 결합

```
array([[10, 11, 12, 13],  
       [14, 15, 16, 17],  
       [ 0,  1,  2,  3]])
```



np.vstack((a, b, d)) # 다차원 배열 3개를 결합

```
array([[10, 11, 12, 13],  
       [14, 15, 16, 17],  
       [ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11],  
       [ 0,  1,  2,  3]])
```

배열의 결합 concatenate, vstack, hstack

- 반면 hstack()은 다음과 같이 axis 0의 차원값이 동일한 때에만 두 벡터를 수평축으로 결합하는 기능이 있다.



```
np.hstack((a, c)) # 다차원 배열 a, c를 수평으로 결합
```

```
array([[10, 11, 12, 13, 0, 1, 2, 3],  
       [14, 15, 16, 17, 4, 5, 6, 7]])
```



```
# a 배열은 형상이 (2, 4), b 배열은 (3, 4)로 수평 결합이 안 됨  
np.hstack((a, b)) # a 배열과 b 배열의 hstack 결합-오류
```

```
ValueError: all the input array dimensions for the concatenation axis must  
match exactly, but along dimension 0, the array at index 0 has size 2 and  
the array at index 1 has size 3
```

배열을 결합하는 r_, c_ 클래스와 column_stack() 함수

- 여러 개의 다차원 배열을 결합하는 또 다른 방법을 살펴보자.
- 이를 위하여 다음과 같은 4개의 다차원 배열을 생성하자.



```
a = np.array([1, 2, 3])  
b = np.array([10, 20, 30])  
c = np.array([[1, 2, 3], [4, 5, 6]])  
d = np.array([[10, 20, 30], [40, 50, 60]])
```

- 이제 a와 b를 연결하는데 concatenate()나 hstack() 대신 r_ 클래스를 이용하여 연결하는 간단한 방법도 있다.
- r_은 큰 괄호 []안에 연결할 원소를 넣거나 다차원 배열을 넣어서 배열을 연결하는 간단하면서도 강력한 클래스이다.
- 이때 연결하는 방향은 첫 번째 축인 axis=0 방향이 된다.

배열을 결합하는 r_, c_ 클래스와 column_stack() 함수



`np.r_[a, b]` # a, b 배열을 결합함 `np.hstack([a, b])`와 같다

```
array([ 1,  2,  3, 10, 20, 30])
```



`np.r_[a, 4, 5, 6, b]` # 중간에 개별 원소 삽입도 가능하다

```
array([ 1,  2,  3,  4,  5,  6, 10, 20, 30])
```



`np.r_[[0]*3, 5, 6]`

```
array([0, 0, 0, 5, 6])
```



`np.r_[c, d]` # 2차원 배열을 `axis=0` 방향으로 결합한다

```
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [10, 20, 30],
       [40, 50, 60]])
```

배열을 결합하는 r_, c_ 클래스와 column_stack() 함수

- 만일 a와 b를 2차원 배열로 만들어서 r_ 클래스로 결합한다면 다음과 같은 결과를 얻을 수 있을 것이다.



```
# a, b를 2차원 배열로 만들어서 axis=0 방향으로 결합  
np.r_[a.reshape(-1,1), b.reshape(-1,1)]
```

```
array([[ 1],  
       [ 2],  
       [ 3],  
      [10],  
      [20],  
      [30]])
```

- 다음은 a, b의 각 원소끼리 묶어서 [1, 10], [2, 20], [3, 30]을 원소로 하는 다차원 배열을 만드는 방법인데, a, b를 2차원 배열로 만들고 나서 axis = 1 축으로 결합하는 방법도 있으나 c_ 클래스를 사용하여 묶어주는 방법과 column_stack() 함수를 사용하여 묶어주는 방법도 있다.

배열을 결합하는 r_, c_ 클래스와 column_stack() 함수



```
np.concatenate((a.reshape(-1, 1), b.reshape(-1, 1)), axis=1)
```

```
array([[ 1, 10],  
       [ 2, 20],  
       [ 3, 30]])
```



```
np.c_[a, b]
```

```
array([[ 1, 10],  
       [ 2, 20],  
       [ 3, 30]])
```



```
np.column_stack([a, b])
```

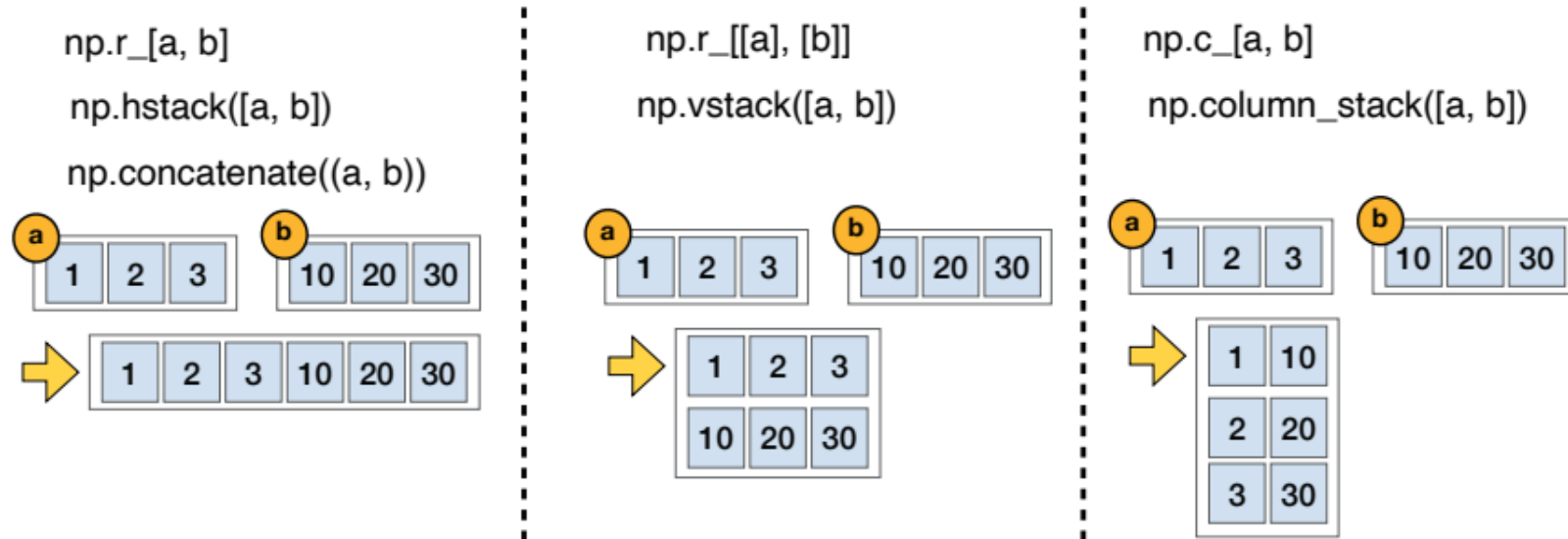
```
array([[ 1, 10],  
       [ 2, 20],  
       [ 3, 30]])
```



```
# c와 d의 전치행렬을 결합시킨다  
np.concatenate((c.T, d.T), axis=1)
```

```
array([[ 1,  4, 10, 40],  
       [ 2,  5, 20, 50],  
       [ 3,  6, 30, 60]])
```

배열을 결합하는 r_, c_ 클래스와 column_stack() 함수



- 넘파이에는 지나치게 많다고 생각이 들 정도의 다양하고도 풍부하면서도 강력한 배열 함수들과 클래스들이 있는데 이 때문에 과학자들의 많은 사랑을 받고 있다.
- 이 밖에도 많은 함수가 제공되고 있으나 이들은 numpy.org의 넘파이 문서를 참고하도록 하자.